

Sisteme de Operare

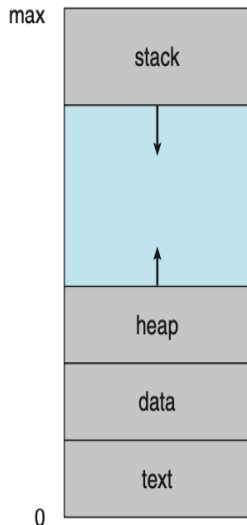
Cap 4. Procese

March 12, 2023

- 1 Conceptul de proces
- 2 Planificarea proceselor
- 3 Operații cu procese
- 4 Comunicații inter-procese
- 5 Comunicația în sistemele client-server

- 1 Conceptul de proces
- 2 Planificarea proceselor
- 3 Operații cu procese
- 4 Comunicații inter-procese
- 5 Comunicația în sistemele client-server

Imaginea procesului în memorie



- **procesul** este un program aflat în execuție; el este unitatea de execuție într-un SO
- **job**: termen istoric, încă utilizat; un singur program rula până la terminare, neîntrerupt
- **program counter**-ul este registrul care stochează instrucțiunea curentă a procesului
- **text section**: codul executabil
- **data section**: variabile statice sau globale
- **heap section**: memorie alocată dinamic, în timpul rulării
- **stack section**: memorie temporară pentru apelul funcțiilor (stochează adresa de revenire din funcție, parametrii funcției, variabilele locale)

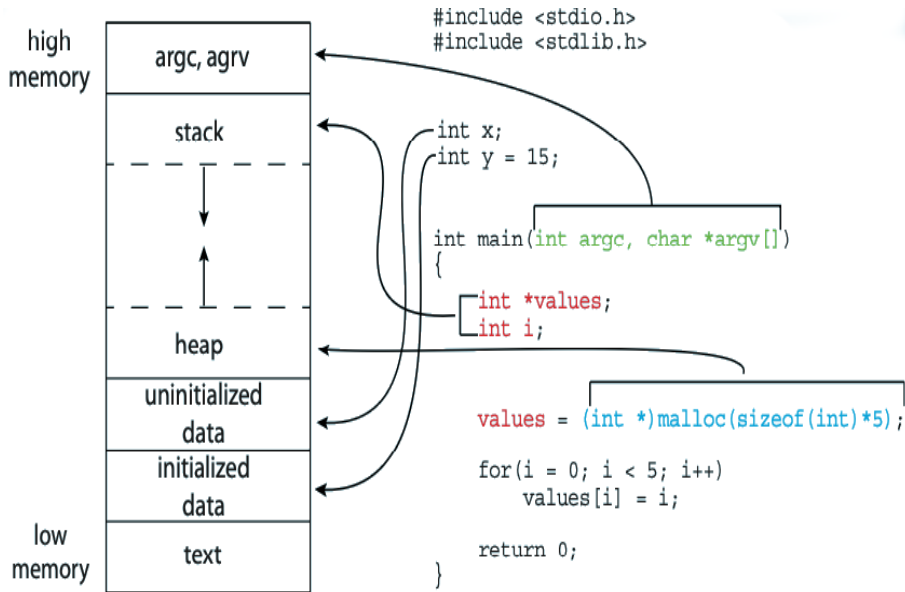
Componența unui proces

- dimensiunile zonelor de date respectiv text (cod) sunt fixe; zona de stivă și cea de heap sunt dinamice, SO are grijă ca ele să nu se suprapună
- la un apel al unei funcții un **activation record** se salvează pe stivă; conține parametrii funcției, variabilele locale ale funcției respectiv adresa de revenire
- același **program**, lansat în execuție de mai multe ori, poate da naștere la mai multe **procese**
- comanda `size` este folosită pentru a vedea dimensiunile secțiunilor fixe
- Block Starting Symbol (BSS) reprezintă zona de date neinițializate, iar `dec` și `hex` sunt exprimările în bazele 10 și 16 a sumei secțiunilor

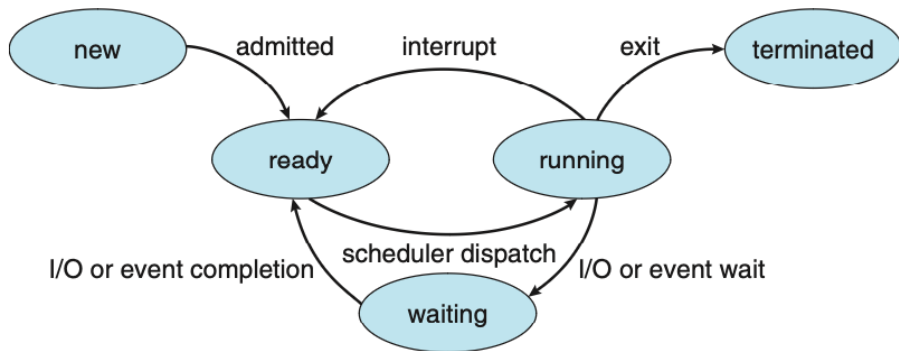
```
[user@fedora lab3]$ size sample
```

text	data	bss	dec	hex	filename
1158	284	8	1450	5aa	sample

Harta memoriei pentru un program C



Stările unui proces



- **new**: procesul este creat
- **ready**: procesul așteaptă să i se aloce CPU
- **running**: se execută instrucțiuni
- **waiting**: procesul așteaptă apariția unui eveniment sau eliberarea unei resurse
- **terminated**: procesul și-a încheiat execuția

Process Control Block

process state
process number
program counter
registers
memory limits
list of open files
...

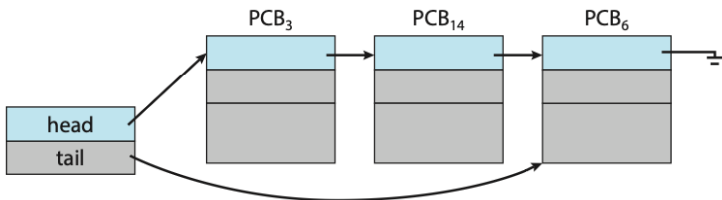
- grupează informațiile procesului
- **Program Counter** indică adresa următoarei instrucțiuni
- **regiștrii CPU** cuprinde informația de stare a procesorului ce execută procesul: regiștri acumulator, de stivă (Stack Pointer), de index, de uz general, sau informații de condiție; starea se salvează la apariția întreruperii de ceas, la planificare ulterioară
- CPU scheduling information (prioritate, cozi)
- memory management information (page tables, segment tables)
- accounting (timp CPU folosit, limite)
- I/O status information (fișiere deschise)

- 1 Conceptul de proces
- 2 Planificarea proceselor**
- 3 Operații cu procese
- 4 Comunicații inter-procese
- 5 Comunicația în sistemele client-server

Planificatorul (scheduler-ul)

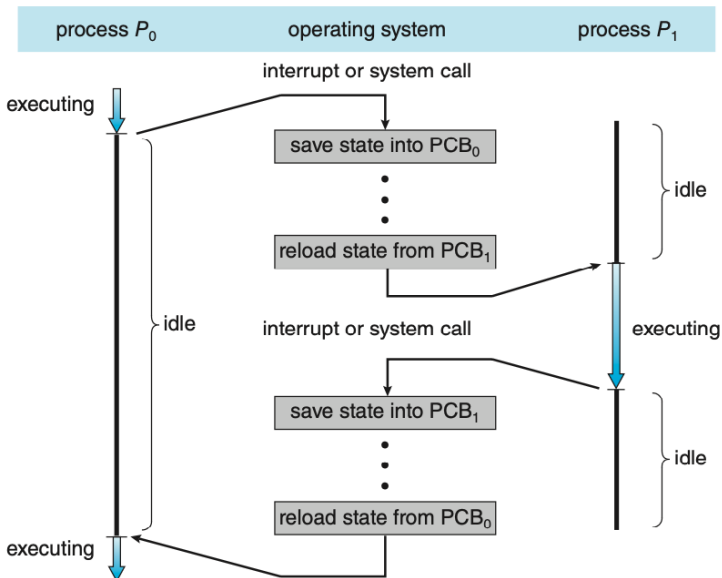
- scopul planificării este maximizarea utilizării CPU
- procesele sunt planificate suficient de des astfel ca utilizatorii să perceapă interacțiunea cu ele
- **planificatorul** (scheduler) selectează următorul proces ce va fi lansat în execuție
- un singur core CPU poate rula un singur proces la un moment dat
- dacă sunt mai multe procese decât core-uri CPU, cele în plus vor trebui să-și aștepte rândul la execuție
- **gradul de multiprogramare** al unui sistem este dat de numărul de procese aflate simultan în execuție

Cozi de procese



- prin crearea proceselor ele sunt puse în coada ready; cozile sunt reprezentate ca liste înlănțuite
- wait queue cuprinde procese ce așteaptă terminarea operațiilor I/O, sau de ex. terminarea proceselor copil pe care le-au inițiat (spawned)
- la expirarea cuantei procesul este plasat în coada ready
- ciclul unui proces prin cozi continuă până la terminarea execuției sale
- în **swapping**, planificatorul evacuează complet procesul din memorie (eliberare de memorie)

Context switch - diagrama de secvență



Context switch (2)

- contextul procesului (starea sa - conținutul regiștrilor) este salvat în PCB când expiră cuanta și este plasat din nou în coada ready
- noului proces planificat i se restaurează starea (i.e. - context switch, schimbarea contextului) și este lansat în execuție pe CPU
- schimbarea de context e dependentă de suportul oferit de procesor, de ex. anumite procesoare oferă mai multe seturi de regiștri, e suficientă doar schimbarea pointer-ului pentru setul curent de regiștri
- la schimbarea contextului, spațiul de adrese al procesului curent trebuie păstrat - realizat prin suportul oferit de memoria virtuală - memoria nu se accesează direct de către proces, ci cu ajutorul unor scheme de adresare indirectă

- 1 Conceptul de proces
- 2 Planificarea proceselor
- 3 Operații cu procese**
- 4 Comunicații inter-procese
- 5 Comunicația în sistemele client-server

Crearea proceselor

- fiecare proces este creat de un proces părinte, astfel el este un proces **child** (copil); creările succesive conduc la un arbore de procese
- fiecare proces este identificat unic în cadrul structurilor kernel-ului de un **process identifier** (PID)
- procesul **systemd** este primul proces creat la boot-area sistemului și are PID-ul 1
- procesele daemon din sistem au numele sufixat cu "d", de ex. **sshd** este procesul pentru servirea clienților care se conectează la sistem folosind protocolul SSH (secure shell)
- listarea tuturor proceselor din sistem se poate face cu comanda `ps -axlf` - toate procesele, nu doar cele personale, toate în sensul de și cele fără terminal asociat, format listă BSD respectiv full-format, cu reliefarea legăturilor de filiație (ex. c-da pstree)

Crearea proceselor (2)

- la crearea unui proces:
 - părintele își continuă execuția în paralel cu copilul
 - părintele așteaptă terminarea execuției copilului
- noul proces copil:
 - poate avea același program și date ca și părintele (este un duplicat al părintelui), sau
 - poate încărca un program diferit
- un nou proces copil este creat cu apelul sistem **fork()**; la revenirea din apel vom avea **două procese** care rulează același cod identic; diferențierea se face prin valoarea returnată: 0 pentru copil (noul proces), respectiv PID-ul procesului creat returnat în părinte
- apelul de sistem **exec()** înlocuiește programul curent cu imaginea binară a programului dat ca parametru și lansează acel program
- apelul de sistem **wait()** determină procesul curent să aștepte terminarea copilului
- procesul copil moștenește privilegiile și atributele de planificare ale părintelui, precum și fișierele deschise

Terminarea procesului

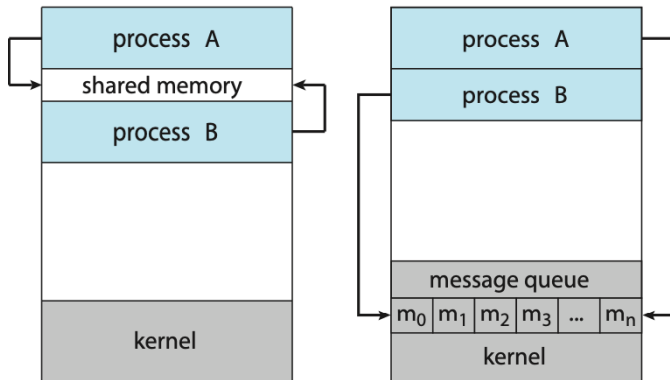
- se realizează prin apelul de sistem **exit()**, cu comunicarea unui cod de eroare către procesul părinte (care se află în apelul wait)
- toate resursele procesului - memoria fizică, memoria virtuală, fișiere deschise, buffere I/O - sunt dealocate de SO
- un proces terminat dar pentru care părintele nu a inspectat codul de terminare prin apelul wait(), se numește proces **zombie**; toate procesele tranzitează scurt prin această stare
- un proces pentru care părintele nu a invocat wait() înainte de terminare se numește proces **orfan**; în sistemele UNIX procesele orfane sunt preluate de **init** sau **systemd** ca părinte
- procesele Android sunt grupate pe clase de importanță (datorită resurselor limitate): procese **foreground**, procese **vizibile** (rezultatul lor e afișat de un proces foreground), procese **service** (activitate înregistrată de utilizator, ex. music streaming), procese **background**, activitatea sa nu e prezentată utilizatorului, procese **empty**, procese fără componente active

- 1 Conceptul de proces
- 2 Planificarea proceselor
- 3 Operații cu procese
- 4 Comunicații inter-procese**
- 5 Comunicația în sistemele client-server

Independență și cooperare

- procesele **independente** nu schimbă informații cu alte procese din sistem
- cele **cooperante** afectează sau pot fi afectate de alte procese din sistem
- primitivile pentru **Inter-Process Communication** (IPC) asigură proceselor: schimbul de informații (procesele interesate de aceleași informații, ex. copy-paste), accelerarea procesării (dacă sistemul are mai multe core-uri CPU), modularitate (funcțiile sunt divizate în procese separate, ex. tab-urile de Chrome)
- două modele IPC fundamentale:
 - **shared memory** (memoria partajată)
 - **message passing** (schimbul de mesaje)
- avantajul abordării IPC multi-proces a unei aplicații - procesele rulează izolat unele de altele, fiecare într-un **sandbox** propriu

Cele două modele IPC

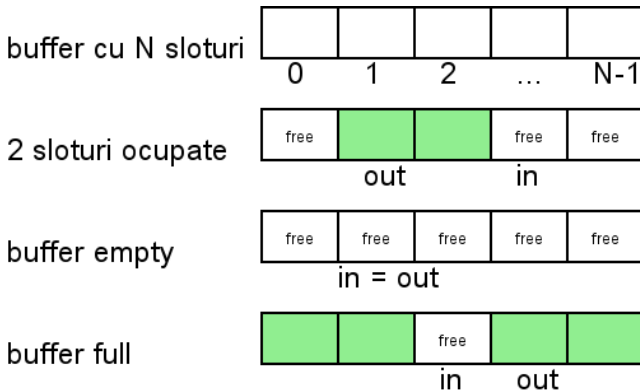


- IPC prin mesaje se folosesc pentru partajarea unor cantități mici de date; potrivite și pentru sistemele distribuite
- comunicația prin shared memory este mai rapidă, sistemul nu face decât să desemneze zonele de memorie, aplicațiile scriu/citesc singure (de) acolo

IPC cu memorie partajată

- memoria partajată (shared) rezidă în spațiul de adrese al procesului; procesele sunt responsabile să se coordoneze la scrierea/citirea în această zonă comună
- problema tipică este problema **producer-consumer**; un proces produce informație consumată de un al doilea proces (ex. compilator-asamblor, server-client etc.)
- implementarea se poate face folosind un **buffer circular** accesibil de către ambele procese
- din cele N sloturi, se pot folosi efectiv doar N-1
- pointer-ul **in** indică spre prima locație liberă iar pointer-ul **out** spre prima locație ocupată

Buffer circular



- condiția de buffer gol este $in == out$
- condiția de buffer plin este $(in + 1) \% N == out$
- se pot folosi toate cele N slot-uri dacă găsim o metodă de a distinge între condiția de coadă plină respectiv coadă goală

Buffer circular - implementarea proceselor

```
// procesul producator
item next_produced;
while (true) {
    /* produce an item in next_produced */
    while (((in + 1) % BUFFER SIZE) == out) ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER SIZE;
}

// procesul consumator
item next_consumed;
while (true) {
    while (in == out) ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER SIZE;
    /* consume the item in next_consumed */
}
```

IPC cu mesaje

- un sistem de comunicație prin mesaje va furniza cel puțin două primitive, `send()` și `receive()`
- mesajele pot avea dimensiune fixă sau variabilă; implementarea comunicației cu mesaje de mărime fixă este mai simplă dpdv. al SO
- între cele două procese trebuie să existe o legătură de comunicație
- **comunicarea directă** presupune că legătura este realizată direct, doar la nivel de pereche de procese; comunicația se face între exact două procese; această modalitate manifestă **simetrie**, i.e. atât sender-ul cât și receiver-ul trebuie să îl refere pe celălalt în apelul de procedură: `send(P, message)` și `receive(Q, message)`, unde P și Q sunt cele 2 procese din pereche
- varianta **asimetrică** a comunicației directe presupune ca doar sender-ul să denumească destinatarul: `send(P, message, dar receive(id, message)`, unde `receive` va întoarce în `id` identificatorul procesului de unde a primit mesajul

Comunicația indirectă

- în comunicația indirectă, mesajele sunt trimise către și recepționate de la porturi (mailboxes)
- un mailbox poate fi privit ca un obiect abstract unde pot fi plasate mesaje, respectiv citite de destinatar; fiecare mailbox are un identificator unic
- deoarece în cadrul perechii atât sender-ul cât și receiver-ul folosesc același ID de mailbox, avem următoarele proprietăți:
 - o legătură poate exista doar dacă mambrii perechii au același mailbox;
 - o legătură poate fi asociată cu mai mult de două procese;
 - în oricare pereche de comunicație, putem avea mai multe legături, fiecare cu mailbox propriu.
- ex. există procesele P_1 , P_2 și P_3 care folosesc mailbox-ul A. Procesul P_1 trimite (send) către A, iar P_2 și P_3 execută receive. Cine primește mesajul?
 - a. impunem ca receive() să nu se execute concomitent sau
 - b. permitem selecția arbitrară ca fie P_2 fir P_3 să poată primi

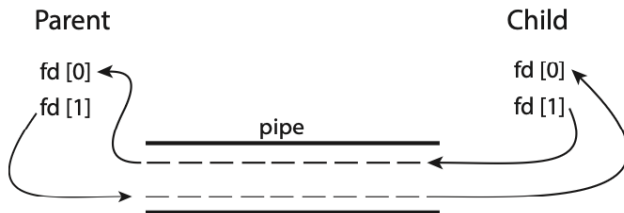
Sincronizarea comunicației

- primitivele de `send()` și `receive()` pot fi fie blocante (blocking, sincrone) sau ne-blocante (nonblocking, asincrone); avem situațiile:
 - **blocking send**: procesul ce trimite este blocat până când mesajul este recepționat de destinatar sau ajunge în mailbox;
 - **nonblocking send**: procesul trimițător trimite mesajul și își rezumă execuția;
 - **blocking receive**: receptorul se blochează până ce un mesaj devine disponibil;
 - **unblocking receive**: dacă nu este disponibil nici un mesaj, receptorul întoarce NULL din apelul `receive()`, și își continuă execuția.
- când atât sender-ul cât și receiver-ul folosesc apeluri `send/receive` blocking, are loc un **rendezvous**; în acest caz problema producător/consumator devine banal de implementat
- mesajele sunt plasate în cozi temporare, alocate de SO; ele pot fi:
 - zero capacity** - sender-ul se va bloca, **bounded capacity** - sender-ul blochează la atingerea capacității sau **unbounded capacity** - sender fără blocking

Pipes (țevi)

- un **pipe** este un buffer FIFO pus la dispoziție de SO, ce permite comunicația între două procese
- următoarele considerente sunt avute în vedere pentru pipes:
 - este comunicația în pipe bidirecțională, sau unidirecțională?
 - dacă se permite comunicația bidirecțională, este aceasta half-duplex (datele pot călători doar pe rând în pipe) sau full-duplex (datele pot călători în ambele direcții în același timp)?
 - se permite comunicația prin pipe doar între procesele părinte-copil sau nu numai?
 - pot pipe-urile comunica prin rețea sau se permite comunicația doar între procesele de pe aceeași mașină?

Pipe-uri tipice



- producătorul scrie la un capăt și consumatorul citește de la celălalt; pipe-urile obișnuite sunt unidirecționale, pentru comunicație în ambele sensuri vom deschide două pipe-uri în perechea de procese
- apelul de sistem este `int pipe(int fd[2])`, iar SO va returna în `fd` descriptorii de fișier; `fd[0]` va fi capătul de citire iar `fd[1]` capătul de scriere
- pipe-urile sunt tratate ca fișiere speciale, ele pot fi folosite cu funcțiile sistem `read()` și `write()`; în mod normal procesul părinte creează pipe-ul și îl folosește ca să comunice cu copilul creat cu `fork()`

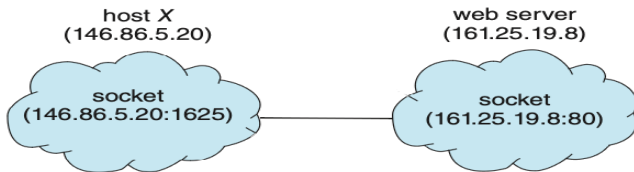
Pipe-uri nominalizate (named pipes, FIFOs)

- pipe-urile nominalizate (care au asociate un nume) permit comunicația între procese care nu au legătură de filiație, respectiv permit comunicația bidirecțională
- sunt create cu apelul de sistem `mkfifo()` și se folosesc cu apelurile `open()`, `read()`, `write()` și `close()`
- un pipe FIFO va continua să existe în SO până va fi șters explicit din sistemul de fișiere
- deși FIFO-urile sunt bidirecționale, comunicația este implementată doar half-duplex; dacă avem nevoie de comunicație bidirecțională implementăm două FIFO-uri
- FIFO-urile Windows au un suport mai bogat: permit comunicație bidirecțională, permit nu doar transfer de bytes ci și de mesaje, respectiv permit comunicația între procese aflate pe mașini diferite
- un exemplu banal de pipe între procese Linux este comanda `bash`
`ls -l | less`

- 1 Conceptul de proces
- 2 Planificarea proceselor
- 3 Operații cu procese
- 4 Comunicații inter-procese
- 5 **Comunicația în sistemele client-server**

Sockets

- un **socket** este o conexiune de rețea prin care putem comunica cu un proces de pe o altă mașină; două procese care comunică de pe mașini diferite vor avea fiecare socket-ul său
- socket-ii folosesc o arhitectură client-server; server-ul așteaptă cereri de la clienți prin **ascultarea** unui anume **port**; odată cererea primită, server-ul **acceptă** conexiunea de la client
- serverele ce implementează servicii specifice (ex. FTP, SSH, HTTP) ascultă pe porturi dedicate, cunoscute (21, 22, 80); toate porturile sub 1024 sunt cunoscute și implementează servicii standard
- la crearea unei cereri de conectare, mașina pe care rulează clientul îi asignează acestuia un port > 1024



Remote Procedures Calls (RPC)

- mecanism folosit pentru a apela, dintr-un proces, proceduri situate pe o altă mașină, conectată prin rețea
- mecanismul RPC se bazează pe comunicația prin mesaje, atât între procese cât și între mașini diferite
- fiecare mesaj RPC este trimis unui proces daemon RPC care ascultă pe un port pe o mașină remote; fiecare mesaj conține un identificator ce specifică numele funcției ce se execută precum și parametrii săi
- funcția se execută pe mașina remote iar rezultatul se împachetează și se trimite înapoi ca mesaj separat
- SO implementează servicii sistem prin care aplicația poate apela proceduri la distanță ca și când acestea ar fi locale; detaliile de implementare sunt împachetate într-o procedură **stub** care există la client
- procedura **stub** va împacheta datele primite ca parametri și trimite către serverul remote pe un anumit port; procesul de împachetare se numește **marshalling**

Remote Procedures Calls (2)

- procesul de marshalling depinde de mașină; pe mașina remote, există o procedură similară care realizează **unmarshalling-ul** parametrilor, respectiv apelul efectiv al procedurii
- pe Windows, codul stub este generat pe baza specificației Microsoft Interface Definition Language (MIDL), folosit la definirea interfețelor dintre programele client și server
- codul de marshalling diferă în funcție de arhitectură; de ex. în convenția **big endian**, stocarea unui întreg pe 32 de biți se face pornind cu cel mai semnificativ byte, pe când în convenția **little endian**, cel mai puțin semnificativ byte este stocat primul
- formatul intermediar este independent de arhitectură; se folosește **eXternal Data Representation** (XDR), marshalling-ul respectiv unmarshalling-ul din stub-urile de la client respectiv server fac automat aceste conversii